



ELDORADO

Trilha de Programação Java

Aula 5.2

Foto de Diego PH,
disponível na Unsplash.
Editada pelo autor.



Faísca

Highlights desafio 4



Atividades de hoje

- ✦ Orientação a objetos com Java
- ✦ Modificadores de Acesso
- ✦ Mão na Massa



POO

***Programação
Orientada a Objetos***



A orientação a objetos é uma abordagem no mundo da programação, que parte da seguinte premissa: precisamos **modelar** o **mundo real** por meio de **objetos** e suas **interações**.

Classe

A **classe** é como a planta baixa para criar um **objeto** casa. É como se fosse o passo a passo que nos diz o que uma casa deve ter. Em outras palavras, define quais **atributos** (variáveis) e **métodos*** (funções) um **objeto** desse tipo terá.



Objetos

Nesse caso, o **objeto** vai ser a casa em si. Vamos usar a **classe** (planta baixa) para criar/construir nossa casa com as características que escolhermos.



Atributos

```
no usages  
private String cor;  
no usages  
private double area;  
no usages  
private int numero;  
no usages  
private int moradores;
```

Essas características que tornam o **objeto** único são os **atributos**. Por pertencerem somente ao objeto, colocamos a palavra **private** na frente.

Métodos

Métodos são as ações que os objetos podem executar, funcionando como os mecanismos de interação entre o **objeto** e o **mundo externo**. Para que essas ações sejam acessíveis, usamos a palavra-chave **'public'** na frente delas.



Métodos são ações (verbos);

```
no usages
public void adicionarMorador() {
    moradores += 1;
}
```

Métodos

Foto de Teemu Paananen,
disponível na Unsplash.
Editada pelo autor.

Regrinha de sintaxe:

- Modificadores de Acesso.
- Tipo de retorno.
- Nome maneiro.
- Parâmetros (Tipo e nome único).

Construtores

Toda classe tem um **construtor** embutido nela e **não** possui parâmetros.

Ele é um método especial para construir o objeto que tem o **mesmo nome da classe** e **não tem retorno**.

Pense como se fosse um ritual de nascimento para os objetos! Mas eles podem ser Ocultos!

Declaração



Aqui estamos dizendo que não vamos precisar passar nenhuma informação na hora de criar um objeto.

Criando o objeto filme usando o construtor padrão.

```
public class Filme {  
    public Filme(){  
    }  
}
```

```
Filme filme = new Filme();
```

Declaração



Nossa classe Filme tem alguns atributos, podemos criar um construtor que nos ajude a preencher esses valores na hora da criação do objeto.

Aqui usamos o construtor com argumentos para criar o objeto passando os respectivos valores.

```
public class Filme { 2 usages

    private String titulo; 1 usage
    private String genero; 1 usage

    public Filme(String tituloParametro, String generoParametro){
        titulo = tituloParametro;
        genero = generoParametro;
    }
}
```

```
Filme filme = new Filme(
    tituloParametro: "Laranja mecânica",
    generoParametro: "Ficção científica"
);
```

Sobrecarga

É a capacidade de termos o mesmo método com assinaturas diferentes, ou seja, diferentes tipos de parâmetros.

```
public void avaliarFilme(String avaliacaoTexto){ no usages
    //.....
}

public void avaliarFilme(String avaliacaoTexto, double nota){ no usages
    //.....
}
```

This

É uma referência ao objeto atual da classe. Geralmente usamos para referenciar os atributos da classe.

```
public void alterarTitulo(String titulo){ no usages  
    this.titulo = titulo;  
}
```

Instância

```
Casa casa1 = new Casa();
```

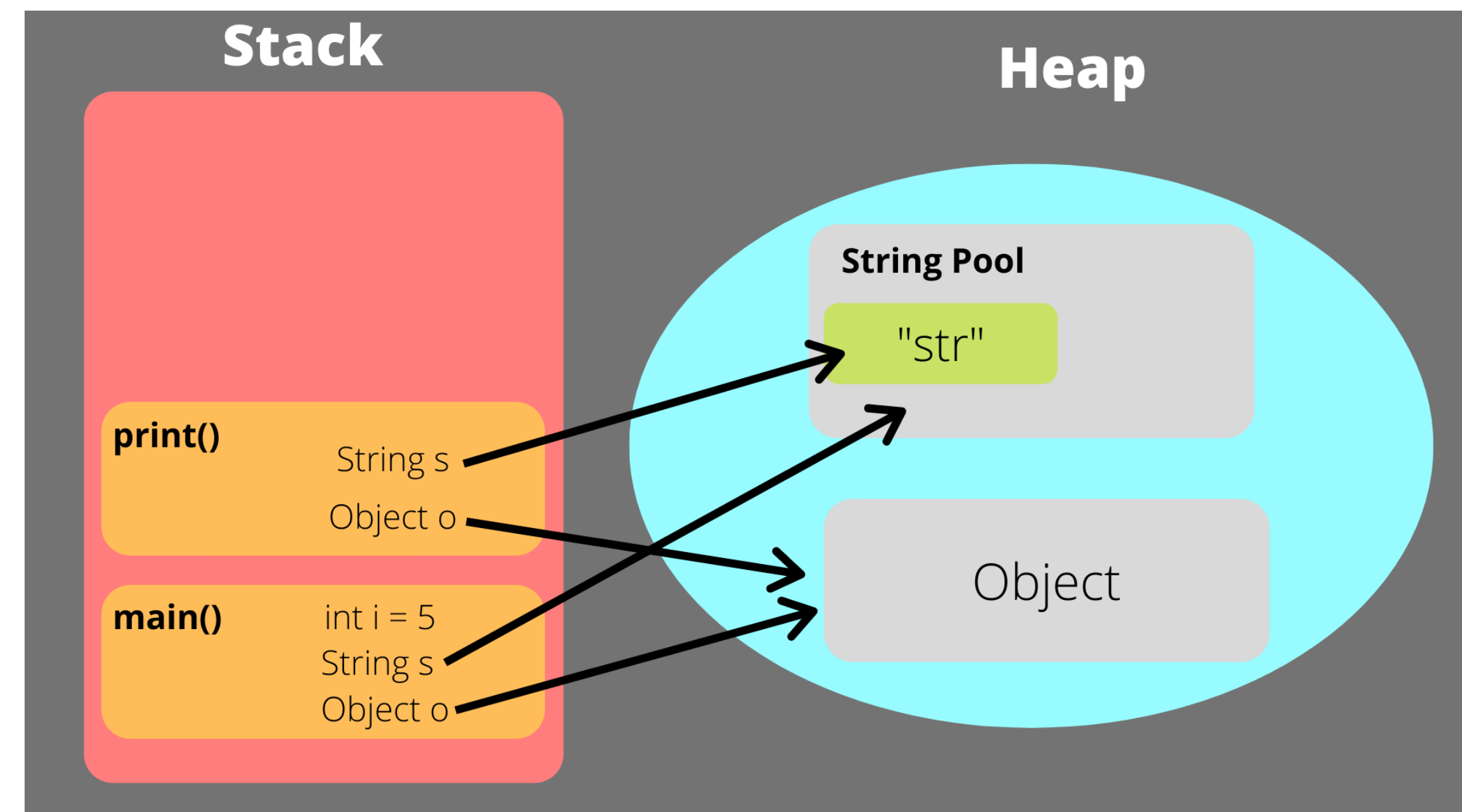
É o nosso objeto criado de fato, pronto para ser utilizado.

Toda vez que utilizamos o **new**, estamos criando uma instância.

Valor x Referência

Stack e Heap

- ✦ São partes da memória RAM com papéis diferentes.
- ✦ Stack: É armazenado as chamadas dos métodos e variáveis primitivas locais.
- Heap: É dinâmica, é onde os dados que precisam persistir além do escopo, ficam.



Pilares POO



Modificador de Acesso

Eles são essenciais por várias razões

◆ Encapsulamento

◆ Abstração

◆ Segurança

public - Acesso Livre! 🎉



Este é como o segurança da balada que permite a entrada de todos. Se marcarmos algo como public, qualquer parte do código pode acessá-lo.

É acessível a todos, meu amigo!

protected - Quase VIP

Se declararmos algo como **protected**, apenas as classes do mesmo pacote terão acesso.

É como uma seção restrita da balada, mas com alguns convidados especiais.

As classes que herdam da classe principal também podem dar uma passadinha nessa área



private - Exclusivo VIP



Se declararmos algo como private, apenas a classe ou o método onde ele está definido podem interagir com ele

É exclusivo, entendeu?

Modificador	Mesma Classe	Mesmo Pacote	Subclasse (Mesmo Pacote)	Subclasse (Outro Pacote)	Outro Pacote
<code>public</code>	Sim	Sim	Sim	Sim	Sim
<code>protected</code>	Sim	Sim	Sim	Sim	Não
Default	Sim	Sim	Sim	Não	Não
<code>private</code>	Sim	Não	Não	Não	Não

Por que não usamos sempre public para tudo?

- ✦ Cria vulnerabilidades de segurança, já que dados sensíveis poderiam ser acessados de qualquer lugar.
- ✦ Dificultar a manutenção, porque ao expor detalhes internos, fica mais difícil modificar a implementação, pode quebrar outras partes do sistema que depende desses detalhes.
- Use com **moderação**

Pensar em uma situação onde o uso de **private** seja essencial?

Um exemplo clássico é em um sistema bancário, onde o **saldo** de uma conta bancária deve ser **private** para que não possa ser alterado diretamente por outras classes. O saldo só deve ser modificado através de métodos controlados, como **depositar()** e **sacar()**, que garantem a integridade das operações.

Getter Setter

Métodos para acessar e alterar o valor dos atributos do objeto.

- Convenção de nomenclatura



```
no usages
public String getCor() {
    return cor;
}

no usages
public void setCor(String cor) {
    this.cor = cor;
}
```

```
no usages
public void pintar(String cor) {
    this.cor = cor;
}
```

Mão Na Massa

POO

Foto de Gaelle Marcel,
disponível na Unsplash.
Editada pelo autor.

Exercício 1

Escolha um objeto da sala, crie a classe com as características e comportamentos que você julgue necessários.

Exercício 2

Crie a classe Carro com no mínimo 4 atributos, será necessário ter um método `exibirDados()`, que ao ser chamado, imprime na tela as informações do carro.

Exercício 3

Aproveite a classe o exercício anterior.

Crie uma classe **Concessionaria** com:

Capacidade de armazenar até **10** Carros.

Um método **adicionarCarro(Carro carro)** para adicionar um carro à lista.

Um método **listarCarros()** que imprime todos os carros cadastrados.

Desafio 5

Crie uma classe **ContaBancaria** com os atributos:

- nome (nome do titular da conta)
- cpf (número de CPF do titular)
- identificadorConta (número de identificação da conta)
- banco (nome do banco)
- endereco (endereço do titular)
- saldo (saldo disponível na conta)
- horarioAtual (um atributo para armazenar o horário atual)

Desafio 5

Implemente os seguintes métodos na classe:

- **saque()**: Realiza um saque da conta, desde que o saldo seja suficiente. Caso contrário, exibe uma mensagem de erro.
- **deposito()**: Realiza um depósito na conta, atualizando o saldo.
- **pix()**: Realiza uma transferência PIX. Caso contrário, exibe uma mensagem de erro.

Desafio 5

- **transferencia():** Realiza uma transferência para outra conta bancária, verificando se o horário atual está dentro do intervalo permitido (8hrs às 19hrs) e se o saldo é suficiente. Caso contrário, exibe uma mensagem de erro.
- **verificarSaldo():** Exibe o saldo atual da conta.
- **verificarHorario():** Exibe o horário atual.
- **verificarInformacoes():** Exibe todas as informações da conta bancária, como nome, CPF, saldo, etc.

Desafio 5

- Implemente um método `alterarEndereco(String novoEndereco)` para atualizar o endereço do titular da conta.
- Adicione uma taxa de manutenção mensal à conta bancária, que seja deduzida automaticamente do saldo no primeiro dia de cada mês.
- Crie um método `calcularJuros(double taxa)` que calcule juros sobre o saldo da conta e os adicione mensalmente.
- Implemente um método `fecharConta()` que encerre a conta, zerando o saldo e não permitindo mais operações.

Até a próxima aula...

19/09/23